

MODULE 2

STACKS AND QUEUES

MODULE-2
Stacks and Queues: Stacks, Stacks Using Dynamic Arrays, Queues, Circular Queues Using Dynamic Arrays, Multiple Stacks and Queues. Stack Applications: Infix to postfix conversion, Infix to prefix conversion, evaluation of postfix expression, recursion.

CHAPTER 3	STACKS AND QUEUES	107
3.1	<u>Stacks</u>	107
3.2	<u>Stacks Using Dynamic Arrays</u>	112
3.3	<u>Queues</u>	114
3.4	<u>Circular Queues Using Dynamic Arrays</u>	119
3.5	<u>A Mazing Problem</u>	120
3.6	<u>Evaluation of Expressions</u>	127
	3.6.1 <u>Expressions</u>	127
	3.6.2 <u>Evaluating Postfix Expressions</u>	
	3.6.3 <u>Infix to Postfix</u>	132
3.7	<u>Multiple Stacks and Queues</u>	139

STACKS AND QUEUES

3.1 STACKS

In this chapter we look at two data types that are frequently found in computer science. These data types, the stack and the queue, are special cases of the more general data type, ordered list, that we discussed in Chapter 2. Recall that $A = a_0, a_1, \dots, a_{n-1}$ is an ordered list of $n \geq 0$ elements. We refer to the a_i as *atoms* or *elements* that are taken from some set. The null or empty list, denoted by $()$, has $n = 0$ elements. In this section we begin by defining the ADT *Stack* and follow with its implementation. Then, we look at the queue.

A *stack* is an ordered list in which insertions (also called pushes and adds) and deletions (also called pops and removes) are made at one end called the *top*. Given a stack $S = (a_0, \dots, a_{n-1})$, we say that a_0 is the bottom element, a_{n-1} is the top element, and a_i is on top of element a_{i-1} , $0 < i < n$. The restrictions on the stack imply that if we add the elements A, B, C, D, E to the stack, in that order, then E is the first element we delete from the stack. Figure 3.1 illustrates this sequence of operations. Since the last element inserted into a stack is the first element removed, a stack is also known as a *Last-In-First-Out (LIFO)* list.

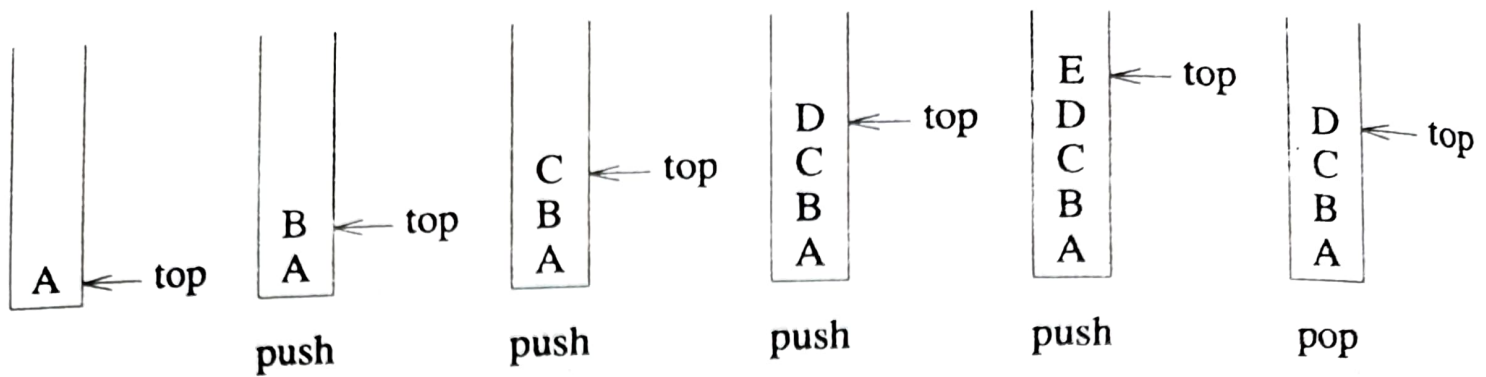


Figure 3.1: Inserting and deleting elements in a stack

Example 3.1 [System stack]: Before we discuss the stack ADT, we look at a special stack, called the **system stack**, that is used by a program at run-time to process function calls. Whenever a function is invoked, the program creates a structure, referred to as an activation record or a stack frame, and places it on top of the system stack. Initially, the activation record for the invoked function contains only a pointer to the previous stack frame and a return address. The previous stack frame pointer points to the stack frame of the invoking function, while the return address contains the location of the statement to be executed after the function terminates. Since only one function executes at any given time, the function whose stack frame is on top of the system stack is chosen. If this function invokes another function, the local variables, except those declared static, and the parameters of the invoking function are added to its stack frame. A new stack frame is then created for the invoked function and placed on top of the system stack. When this function terminates, its stack frame is removed and the processing of the invoking function, which is again on top of the stack, continues. A simple example illustrates this process.

Assume that we have a main function that invokes function *a1*. Figure 3.2(a) shows the system stack before *a1* is invoked; Figure 3.2(b) shows the system stack after *a1* has been invoked. Frame pointer *fp* is a pointer to the current stack frame. The system also maintains separately a stack pointer, *sp*, which we have not illustrated.

Since all functions are stored similarly in the system stack, it makes no difference if the invoking function calls itself. That is, a recursive call requires no special strategy; the run-time program simply creates a new stack frame for each recursive call. However, recursion can consume a significant portion of the memory allocated to the system stack; it could consume the entire available memory. □

Our discussion of the system stack suggests several operations that we include in the ADT specification (ADT 3.1).

The easiest way to implement this ADT is by using a one-dimensional array, say,

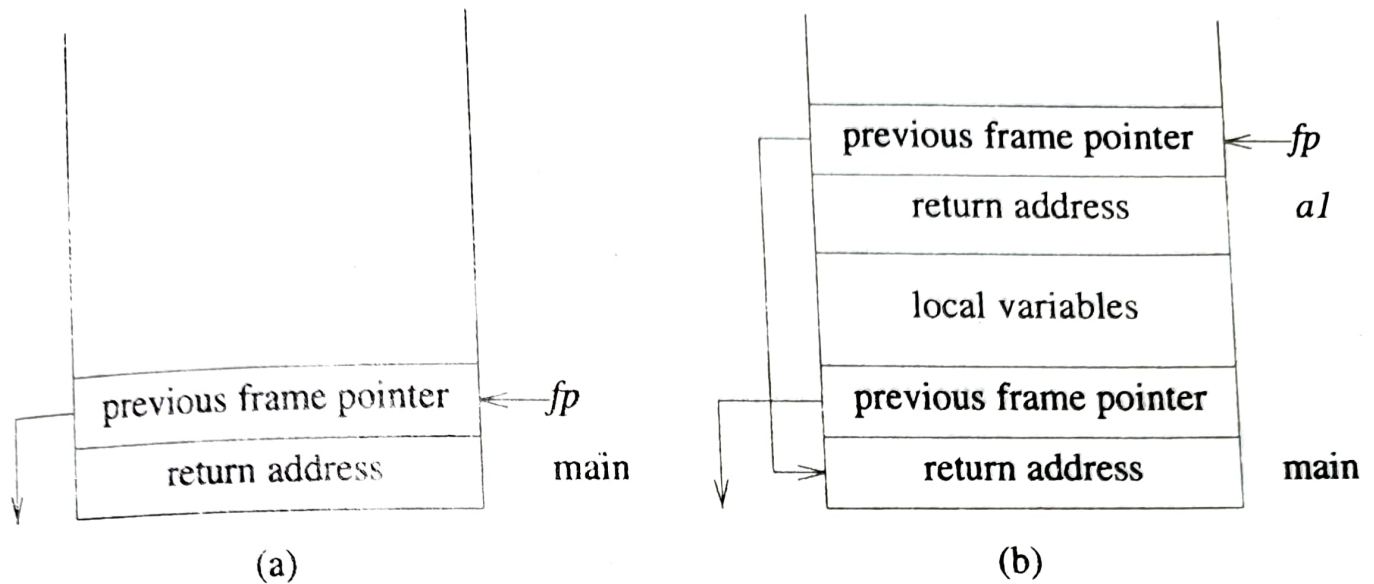


Figure 3.2: System stack after function call

$stack[MAX_STACK_SIZE]$, where MAX_STACK_SIZE is the maximum number of entries. The first, or bottom, element of the stack is stored in $stack[0]$, the second in $stack[1]$, and the i th in $stack[i-1]$. Associated with the array is a variable, top , which points to the top element in the stack. Initially, top is set to -1 to denote an empty stack. Given this representation, we can implement the operations in ADT 3.1 as follows. Notice that we have specified that *element* is a structure that consists of only a *key* field. Ordinarily, we would not create a structure with a single field. However, we use *element* in this and subsequent chapters as a template whose fields we may add to or modify to meet the requirements of our application.

```
Stack CreateS(maxStackSize) ::=
    #define MAX_STACK_SIZE 100 /* maximum stack size */
    typedef struct {
        int key;
        /* other fields */
    } element;
    element stack[MAX_STACK_SIZE];
    int top = -1;
```

```
Boolean IsEmpty(Stack) ::= top < 0;
```

```
Boolean IsFull(Stack) ::= top >= MAX_STACK_SIZE-1;
```


ADT Stack is

objects: a finite ordered list with zero or more elements.

functions:

for all $stack \in Stack$, $item \in element$, $maxStackSize \in$ positive integer

$Stack\ CreateS(maxStackSize) ::=$

create an empty stack whose maximum size is $maxStackSize$

$Boolean\ IsFull(stack, maxStackSize) ::=$

if (number of elements in $stack == maxStackSize$)

return *TRUE*

else return *FALSE*

$Stack\ Push(stack, item) ::=$

if ($IsFull(stack)$) $stackFull$

else insert $item$ into top of $stack$ and return

$Boolean\ IsEmpty(stack) ::=$

if ($stack == CreateS(maxStackSize)$)

return *TRUE*

else return *FALSE*

$Element\ Pop(stack) ::=$

if ($IsEmpty(stack)$) return

else remove and return the element at the top of the stack.

ADT 3.1: Abstract data type Stack★

The *IsEmpty* and *IsFull* operations are simple, and we will implement them directly in the *push* (Program 3.1) and *pop* (Program 3.2) functions. Each of these functions assumes that the variables *stack* and *top* are global. The functions are short and require little explanation. Function *push* checks to see if the stack is full. If it is, it calls *stackFull* (Program 3.3), which prints an error message and terminates execution. When the stack is not full, we increment *top* and assign *item* to $stack[top]$. Implementation of the pop operation parallels that of the push operation. The code of Program 3.2 assumes that the *stackEmpty* function prints an error message and returns an item of type *element* with a *key* field that contains an error code. Typical function calls would be $push(item)$; and $item = pop()$;

Note:

```
void push(element item)
{
    /* add an item to the global stack */
    if (top >= MAX_STACK_SIZE-1)
        stackFull();
    stack[++top] = item;
}
```

Program 3.1: Add an item to a stack

```
element pop()
{
    /* delete and return the top element from the stack */
    if (top == -1)
        return stackEmpty(); /* returns an error key */
    return stack[top--];
}
```

Program 3.2: Delete from a stack

```
void stackFull()
{
    fprintf(stderr, "Stack is full, cannot add element");
    exit(EXIT_FAILURE);
}
```

Program 3.3: Stack full

3.2 STACKS USING DYNAMIC ARRAYS

A shortcoming of the stack implementation of the preceding section is the need to know, at compile time, a good bound (*MAX-STACK-SIZE*) on how large the stack will become. We can overcome this shortcoming by using a dynamically allocated array for the elements and then increasing the size of this array as needed. The following implementation of *CreateS*, *IsEmpty*, and *IsFull* uses a dynamically allocated array *stack* whose initial capacity (i.e., maximum number of stack elements that may be stored in the array) is 1. Specific applications may dictate other choices for the initial capacity.

```
Stack CreateS() ::= typedef struct {  
    int key;  
    /* other fields */  
} element;  
element *stack;  
MALLOC(stack, sizeof(*stack));  
int capacity = 1;  
int top = -1;
```



```
Boolean IsEmpty(Stack) ::= top < 0;
```

```
Boolean IsFull(Stack) ::= top >= capacity-1;
```

While we must alter the code for the *push* function (Program 3.1) to use the new test for a full stack (replace *MAX_STACK_SIZE* with *capacity*), the code for the *pop* function (Program 3.2) is unchanged. Additionally, the code for *stackFull* is changed. The new code for *stackFull* attempts to increase the capacity of the array *stack* so that we can add an additional element to the stack. Before we can increase the capacity of an array, we must decide what the new capacity should be. In *array doubling*, we double array capacity whenever it becomes necessary to increase the capacity of an array. Program 3.4 gives the code for *stackFull* when array doubling is used.

```
void stackFull()
{
    REALLOC(stack, 2 * capacity * sizeof(*stack))
    capacity *= 2;
}
```

Program 3.4: Stack full with array doubling

Although it may appear that a lot of time is spent doubling the capacity of *stack*, this is actually not the case. In the worst case, the *realloc* function needs to allocate $2 * \text{capacity} * \text{sizeof}(*\text{stack})$ bytes of memory and copy $\text{capacity} * \text{sizeof}(*\text{stack})$ bytes of memory from the old array into the new one. Under the assumptions that memory may be allocated in $O(1)$ time and that a stack element can be copied in $O(1)$ time, the time required by array doubling is $O(\text{capacity})$. Initially, *capacity* is 1. Suppose that when we are done with all the stack pushes we wish to perform, *capacity* is 2^k for some $k, k > 0$. The total time spent over all array doublings is $O(\sum_{i=1}^k 2^i) = O(2^{k+1}) = O(2^k)$. Since the total number of pushes is more than 2^{k-1} (otherwise the array capacity would not have been doubled from 2^{k-1} to 2^k), the total time spent in array doubling is $O(n)$, where n is the total number of pushes. Hence, even with the time spent on array doubling added in, the total run time of *push* over all n pushes is $O(n)$. Notice that this conclusion remains valid whenever *stackFull* resizes the stack array by a factor $c > 1$ ($c=2$ in Program 3.4).

3.3 QUEUES

A *queue* is an ordered list in which insertions (also called additions, puts, and pushes) and deletions (also called removals and pops) take place at different ends. The end at which new elements are added is called the *rear*, and that from which old elements are deleted is called the *front*. The restrictions on a queue imply that if we insert A, B, C, D, and E in that order, then A is the first element deleted from the queue. Figure 3.4 illustrates this sequence of events. Since the first element inserted into a queue is the first element removed, queues are also known as *First-In-First-Out (FIFO)* lists. The ADT specification of the queue appears in ADT 3.2.

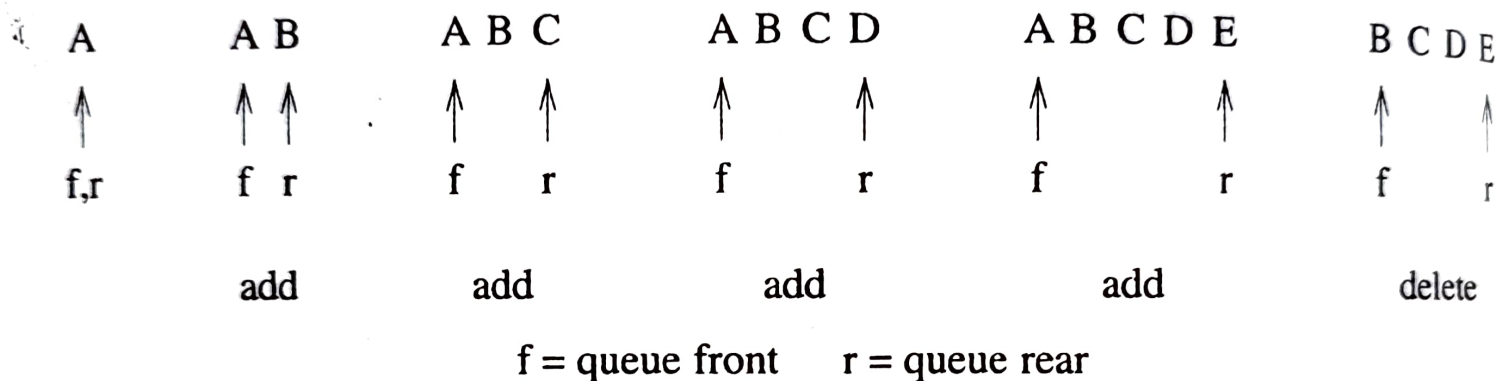


Figure 3.4: Inserting and deleting elements in a queue

The representation of a queue in sequential locations is more difficult than that of the stack. The simplest scheme employs a one-dimensional array and two variables, *front* and *rear*. Given this representation, we can define the queue operations in ADT 3.2 as:

ADT *Queue* is

objects: a finite ordered list with zero or more elements.

functions:

for all *queue* $\in$ *Queue*, *item* $\in$ *element*, *maxQueueSize* $\in$ positive integer

Queue **CreateQ**(*maxQueueSize*) ::=

create an empty queue whose maximum size is *maxQueueSize*

Boolean **IsFullQ**(*queue*, *maxQueueSize*) ::=

if (number of elements in *queue* == *maxQueueSize*)

return *TRUE*

else return *FALSE*

Queue **AddQ**(*queue*, *item*) ::=

if (**IsFullQ**(*queue*)) *queueFull*

else insert *item* at rear of *queue* and return *queue*

Boolean **IsEmptyQ**(*queue*) ::=

if (*queue* == **CreateQ**(*maxQueueSize*))

return *TRUE*

else return *FALSE*

Element **DeleteQ**(*queue*) ::=

if (**IsEmptyQ**(*queue*)) return

else remove and return the *item* at front of *queue*.

ADT 3.2: Abstract data type *Queue*

Queue **CreateQ**(*maxQueueSize*) ::=

```
#define MAX_QUEUE_SIZE 100 /* maximum queue size */
```

```
typedef struct {
```

```
    int key;
```

```
    /* other fields */
```

```
    } element;
```

```
element queue[MAX_QUEUE_SIZE];
```

```
int rear = -1;
```

```
int front = -1;
```

Boolean **IsEmptyQ**(*queue*) ::= front == rear

Boolean **IsFullQ**(*queue*) ::= rear == MAX_QUEUE_SIZE-1

Since the *IsEmptyQ* and *IsFullQ* operations are quite simple, we again implement them directly in the *addq* (Program 3.5) and *deleteq* (Program 3.6) functions. The implementation of *queueFull* is similar to that of *stackFull* (Program 3.3). Functions *addq* and *deleteq* are structurally similar to *push* and *pop* on stacks. While the stack uses the

116 Stacks And Queues

variable *top* in both *push* and *pop*, the queue increments *rear* in *addq* and *front* in *deleteq*. Typical function calls would be *addq (item)*; and *item = deleteq ()*;

```
void addq(element item)
{
    /* add an item to the queue */
    if (rear == MAX_QUEUE_SIZE-1)
        queueFull();
    queue[++rear] = item;
}
```

Program 3.5: Add to a queue

```
element deleteq()
{
    /* remove element at the front of the queue */
    if (front == rear)
        return queueEmpty(); /* return an error key */
    return queue[++front];
}
```

Program 3.6: Delete from a queue

This sequential representation of a queue has pitfalls that are best illustrated by an example.

Example 3.2 [Job scheduling]: Queues are frequently used in computer programming, and a typical example is the creation of a job queue by an operating system. If the operating system does not use priorities, then the jobs are processed in the order they enter the system. Figure 3.5 illustrates how an operating system might process jobs if it used a sequential representation for its queue.

It should be obvious that as jobs enter and leave the system, the queue gradually shifts to the right. This means that eventually the rear index equals *MAX_QUEUE_SIZE - 1*, suggesting that the queue is full. In this case, *queueFull* should move the entire queue to the left so that the first element is again at *queue [0]* and *front* is at *- 1*. It should also recalculate *rear* so that it is correctly positioned. Shifting an array is very time-consuming, particularly when there are many elements in it. In fact, *queueFull* has a worst case complexity of $O(\text{MAX_QUEUE_SIZE})$. □

We can obtain a more efficient queue representation if we permit the queue to wrap around the end of the array. At this time it is convenient to think of the array

front	rear	Q[0]	Q[1]	Q[2]	Q[3]	Comments
-1	-1					queue is empty
-1	0	J1				Job 1 is added
-1	1	J1	J2			Job 2 is added
-1	2	J1	J2	J3		Job 3 is added
0	2		J2	J3		Job 1 is deleted
1	2			J3		Job 2 is deleted

Figure 3.5: Insertion and deletion from a sequential queue

positions as arranged in a circle (Figure 3.6) rather than in a straight line (Figure 3.4). In Figure 3.6, we have changed the convention for the variable *front*. This variable now points one position counterclockwise from the location of the front element in the queue. The convention for *rear* is unchanged. This change simplifies the codes slightly.

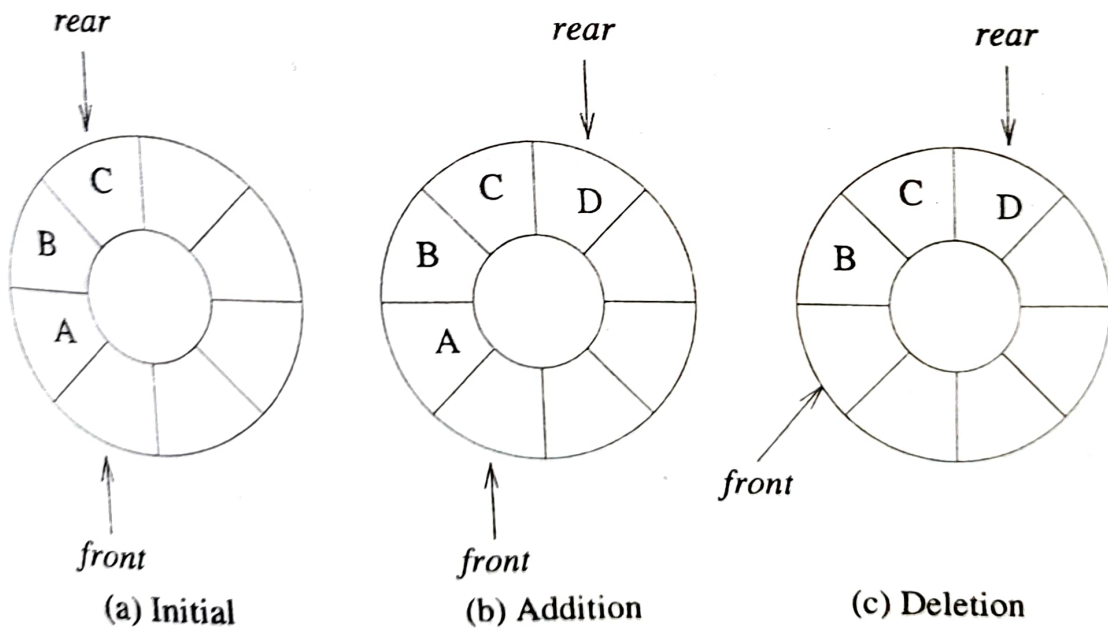


Figure 3.6: Circular queue

When the array is viewed as a circle, each array position has a next and a previous position. The position next to position $\text{MAX_QUEUE_SIZE} - 1$ is 0, and the position that precedes 0 is $\text{MAX_QUEUE_SIZE} - 1$. When the queue *rear* is at $\text{MAX_QUEUE_SIZE} - 1$, the next element is put into position 0. To work with a

circular queue, we must be able to move the variables *front* and *rear* from their current position to the next position (clockwise). This may be done using code such as

```
if (rear == MAX-QUEUE-SIZE - 1) rear = 0;
else rear++;
```

Using the modulus operator, which computes remainders, this code is equivalent to $(rear+1) \% MAX_QUEUE_SIZE$. With our conventions for *front* and *rear*, we see that the front element of the queue is located one position clockwise from *front* and the rear element is at position *rear*.

To determine a suitable test for an empty queue, we experiment with the queues of Figure 3.6. To delete an element, we advance *front* one position clockwise and to add an element, we advance *rear* one position clockwise and insert at the new position. If we perform 3 deletions from the queue of Figure 3.6(c) in this fashion, we will see that the queue becomes empty and that *front* = *rear*. When we do 5 additions to the queue of Figure 3.6(b), the queue becomes full and *front* = *rear*. So, we cannot distinguish between an empty and a full queue. To avoid the resulting confusion, we shall increase the capacity of a queue just before it becomes full. Consequently, *front* == *rear* iff the queue is empty. The initial value for both *front* and *rear* is 0. Progrms 3.7 and 3.8, respectively, given the codes to add and delete. The code for *queueFull* is similar to that of the *stackFull* code of Program 3.3.

```
void addq(element item)
{ /* add an item to the queue */
    rear = (rear+1) % MAX-QUEUE-SIZE;
    if (front == rear)
        queueFull(); /* print error and exit */
    queue[rear] = item;
}
```

Program 3.7: Add to a circular queue

Observe that the test for a full queue in *addq* and the test for an empty queue in *deleteq* are the same. In the case of *addq*, however, when *front* = *rear* is evaluated and found to be true, there is actually one space free (*queue[rear]*) since the first element in the queue is not at *queue[front]* but is one position clockwise from this point. As remarked earlier, if we insert an item here, then we will not be able to distinguish between the cases of full and empty, since the insertion would leave *front* equal to *rear*. To avoid this we signal *queueFull*, thus permitting a maximum of $MAX_QUEUE_SIZE - 1$ rather than MAX_QUEUE_SIZE elements in the queue at any time. We leave the implementation of *queueFull* as an exercise.


```
element deleteq()  
{/* remove front element from the queue */  
    element item;  
    if (front == rear)  
        return queueEmpty(); /* return an error key */  
    front = (front+1) % MAX-QUEUE-SIZE;  
    return queue[front];  
}
```

3.4 CIRCULAR QUEUES USING DYNAMICALLY ALLOCATED ARRAYS

Suppose that a dynamically allocated array is used to hold the queue elements. Let *capacity* be the number of positions in the array *queue*. To add an element to a full queue, we must first increase the size of this array using a function such as *realloc*. As with dynamically allocated stacks, we use array doubling. However, it isn't sufficient to simply double array size using *realloc*. Consider the full queue of Figure 3.7(a). This

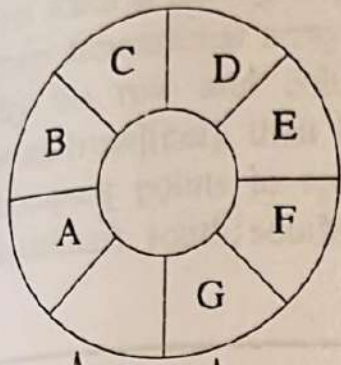
120 Stacks And Queues

figure shows a queue with seven elements in an array whose capacity is 8. To visualize array doubling when a circular queue is used, it is better to flatten out the array as in Figure 3.7(b). Figure 3.7(c) shows the array after array doubling by *realloc*.

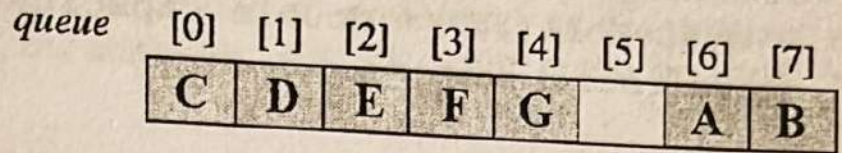
To get a proper circular queue configuration, we must slide the elements in the right segment (i.e., elements *A* and *B*) to the right end of the array as in Figure 3.7(d). The array doubling and the slide to the right together copy at most $2 * \text{capacity} - 2$ elements. The number of elements copied can be limited to $\text{capacity} - 1$ by customizing the array doubling code so as to obtain the configuration of Figure 3.7(e). This configuration may be obtained as follows:

- (1) Create a new array *newQueue* of twice the capacity.
- (2) Copy the second segment (i.e., the elements *queue*[*front*+1] through *queue*[*capacity*-1]) to positions in *newQueue* beginning at 0.
- (3) Copy the first segment (i.e., the elements *queue*[0] through *queue*[*rear*]) to positions in *newQueue* beginning at *capacity*-*front*-1.

Program 3.9 gives the code to add to a circular queue using a dynamically allocated array. Program 3.10 gives the code for *queueFull*. The function *copy(a,b,c)* copies elements from locations *a* through *b*-1 to locations beginning at *c*. Program 3.10 obtains the configuration of Figure 3.7(e).

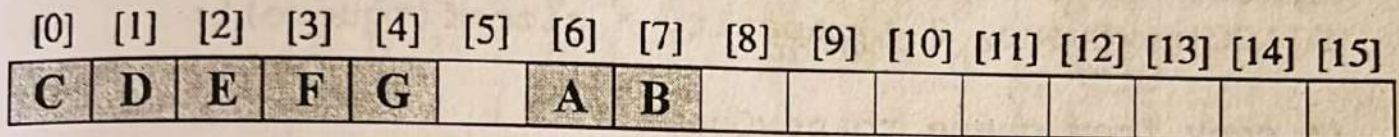


(a) A full circular queue



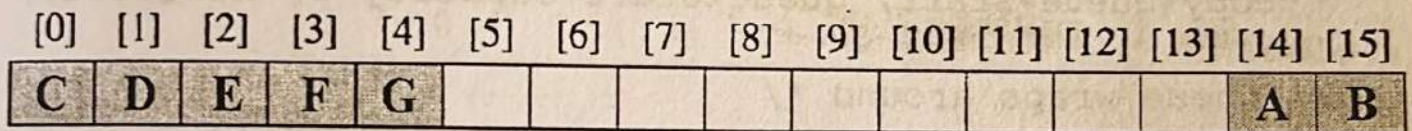
$front = 5, rear = 4$

(b) Flattened view of circular full queue



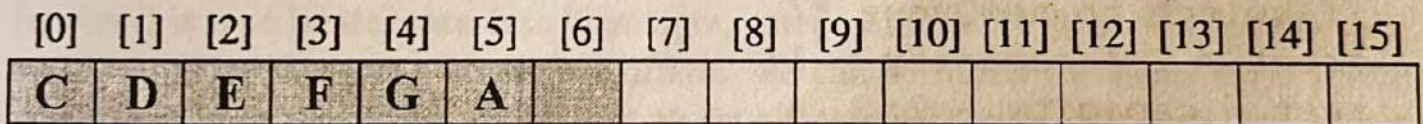
$front = 5, rear = 4$

(c) After array doubling



$front = 13, rear = 4$

(d) After shifting right segment



$front = 15, rear = 6$

(e) Alternative configuration

Figure 3.7: Doubling queue capacity

```

void addq(element item)
{
    /* add an item to the queue */
    rear = (rear+1) % capacity;
    if (front == rear)
        queueFull(); /* double capacity */
    queue[rear] = item;
}

```

Program 3.9: Add to a circular queue

```

void queueFull()
{
    /* allocate an array with twice the capacity */
    element* newQueue;
    MALLOC(newQueue, 2 * capacity * sizeof(*queue));

    /* copy from queue to newQueue */
    int start = (front+1) % capacity;
    if (start < 2)
        /* no wrap around */
        copy(queue+start, queue+start+capacity-1, newQueue);
    else
        /* queue wraps around */
        copy(queue+start, queue+capacity, newQueue);
        copy(queue, queue+rear+1, newQueue+capacity-start);
    }

    /* switch to newQueue */
    front = 2 * capacity - 1;
    rear = capacity - 2;
    capacity *= 2;
    free(queue);
    queue = newQueue;
}

```

Program 3.10: Doubling queue capacity

3.6 EVALUATION OF EXPRESSIONS

3.6.1 Expressions

The representation and evaluation of expressions is of great interest to computer scientists. As programmers, we write complex expressions such as:

$$((rear + 1 == front) \parallel ((rear == MAX_QUEUE_SIZE - 1) \&\& ! front)) \quad (3.1)$$

or complex assignment statements such as:

$$x = a / b - c + d * e - a * c \quad (3.2)$$

If we examine expression (3.1), we notice that it contains operators (`==`, `+`, `-`, `||`, `&&`, `!`), operands (`rear`, `front`, `MAX-QUEUE-SIZE`), and parentheses. The same is true of the statement (3.2), although the operands and operators have changed, and there are no parentheses.

The first problem with understanding the meaning of these or any other expressions and statements is figuring out the order in which the operations are performed. For instance, assume that $a = 4$, $b = c = 2$, $d = e = 3$ in statement (3.2). We want to find the value of x . Is it

$$\begin{aligned}
 & ((4/2) - 2) + (3 * 3) - (4 * 2) \\
 & = 0 + 9 - 8 \\
 & = 1
 \end{aligned}$$

or

$$\begin{aligned}
 & (4/(2-2+3)) * (3-4) * 2 \\
 & = (4/3) * (-1) * 2 \\
 & = -2.66666 \dots
 \end{aligned}$$

Most of us would pick the first answer because we know that division is carried out before subtraction, and multiplication before addition. If we wanted the second answer, we would have written (3.2) differently, using parentheses to change the order of evaluation:

$$x = ((a/(b-c+d)) * (e-a)) * c \quad (3.3)$$

Within any programming language, there is a precedence hierarchy that determines the order in which we evaluate operators. Figure 3.12 contains the precedence hierarchy for C. We have arranged the operators from highest precedence to lowest. Operators with the same precedence appear in the same box. For instance, the highest precedence operators are function calls, array elements, and structure or union members, while the comma operator has the lowest precedence. Operators with highest precedence are evaluated first. The associativity column indicates how we evaluate operators with the same precedence. For instance, the multiplicative operators have left-to-right associativity. This means that the expression $a*b/c\%d/e$ is equivalent to $((((a*b)/c)\%d)/e)$. In other words, we evaluate the operator that is furthest to the left first. With right associative operators of the same precedence, we evaluate the operator furthest to the right first. Parentheses are used to override precedence, and expressions are always evaluated from the innermost parenthesized expression first. } rule

3.6.2 Evaluating Postfix Expressions

The standard way of writing expressions is known as infix notation because in it we place a binary operator in-between its two operands. We have used this notation for all of the expressions written thus far. Although infix notation is the most common way of writing expressions, it is not the one used by compilers to evaluate expressions. Instead compilers typically use a parenthesis-free notation referred to as postfix. In this notation, each operator appears after its operands. Figure 3.13 contains several infix expressions and their postfix equivalents. *

Before writing a function that translates expressions from infix to postfix, we tackle the easier task of evaluating postfix expressions. This evaluation process is much simpler than the evaluation of infix expressions because there are no parentheses to consider. To evaluate an expression we make a single left-to-right scan of it. We place the operands on a stack until we find an operator. We then remove, from the stack, the

Token	Operator	Precedence ¹	Associativity
() [] → .	function call array element struct or union member	17	left-to-right
-- ++	increment, decrement ²	16	left-to-right
-- ++ ! ~ - + & * sizeof	decrement, increment ³ logical not one's complement unary minus or plus address or indirection size (in bytes)	15	right-to-left
(type)	type cast	14	right-to-left
* / %	multiplicative	13	left-to-right
+ -	binary add or subtract	12	left-to-right
<< >>	shift	11	left-to-right
> >= < <=	relational	10	left-to-right
== !=	equality	9	left-to-right
&	bitwise and	8	left-to-right
^	bitwise exclusive or	7	left-to-right
	bitwise or	6	left-to-right
&&	logical and	5	left-to-right
	logical or	4	left-to-right
?:	conditional	3	right-to-left
= += -= /= *= %= <<= >>= &= ^= =	assignment	2	right-to-left
,	comma	1	left-to-right

1. The precedence column is taken from Harbison and Steele.

2. Postfix form

3. Prefix form

Figure 3.12: Precedence hierarchy for C

correct number of operands for the operator, perform the operation, and place the result back on the stack. We continue in this fashion until we reach the end of the expression. We then remove the answer from the top of the stack. Figure 3.14 shows this processing when the input is the nine character string $6\ 2/3-4\ 2*+$.

Infix	Postfix
$2+3*4$	$2\ 3\ 4*+$
$a*b+5$	$ab*5+$
$(1+2)*7$	$1\ 2+7*$
$a*b/c$	$ab*c/$
$((a/(b-c+d))*(e-a))*c$	$abc-d+/ea-*c*$
$a/b-c+d*e-a*c$	$ab/c-de*+ac*-$

Figure 3.13: Infix and postfix notation

Token	[0]	Stack [1]	[2]	Top
6	6			0
2	6	2		1
/	6/2			0
3	6/2	3		1
-	6/2-3			0
4	6/2-3	4		1
2	6/2-3	4	2	2
*	6/2-3	4*2		1
+	6/2-3+4*2			0

Figure 3.14: Postfix evaluation

We now consider the representation of both the stack and the expression. To simplify our task we assume that the expression contains only the binary operators $+$, $-$, $*$, $/$, and $\%$ and that the operands in the expression are single digit integers as in Figure 3.14. This permits us to represent the expression as a character array. The operands are stored on a stack of type `int` until they are needed. We may use either of the representations of Sections 3.1 and 3.2. It is convenient to define the enumerated type *precedence*, which lists the operators by mnemonics, as below:


```
typedef enum {lparen, rparen, plus, minus, times, divide,
              mod, eos, operand} precedence;
```

Although we will use it to process tokens (operators, operands, and parentheses) in this example, its real importance becomes evident when we translate infix expressions into postfix ones. Besides the usual operators, the enumerated type also includes an end-of-string (*eos*) operator.

The function *eval* (Program 3.13) contains the code to evaluate a postfix expression. Since an operand (*symbol*) is initially a character, we must convert it into a single digit integer. We use the statement, *symbol* - '0', to accomplish this task. The statement takes the ASCII value of *symbol* and subtracts the ASCII value of '0', which is 48, from it. For example, suppose *symbol* = '1'. The character, '1', has an ASCII value of 49. Therefore, the statement *symbol* - '0' produces as result the number 1.

We use an auxiliary function, *getToken* (Program 3.14), to obtain tokens from the expression string. If the token is an operand, we convert it to a number and add it to the stack. Otherwise, we remove two operands from the stack, perform the specified operation, and place the result back on the stack. When we have reached the end of expression, we remove the result from the stack.

3.6.3 Infix to Postfix

We can describe an algorithm for producing a postfix expression from an infix one as follows:

- (1) Fully parenthesize the expression.
- (2) Move all binary operators so that they replace their corresponding right parentheses.
- (3) Delete all parentheses.

For example, $a/b - c + d * e - a * c$ when fully parenthesized becomes:

$$((((a/b) - c) + (d * e)) - a * c))$$

Performing steps 2 and 3 gives:

$$ab/c - de * + ac * -$$

Although this algorithm works well when done by hand, it is inefficient on a computer because it requires two passes. The first pass reads the expression and parenthesizes it, while the second moves the operators. Since the order of operands is the same in infix and postfix, we can form the postfix equivalent by scanning the infix expression left-to-right. During this scan, operands are passed to the output expression as they are encountered. However, the order in which the operators are output depends


```

int eval(void)
{
    /* evaluate a postfix expression, expr, maintained as a
       global variable. '\0' is the end of the expression.
       The stack and top of the stack are global variables.
       getToken is used to return the token type and
       the character symbol. Operands are assumed to be single
       character digits */
    precedence token;
    char symbol;
    int op1, op2;
    int n = 0; /* counter for the expression string */
    int top = -1;
    token = getToken(&symbol, &n);
    while (token != eos) {
        if (token == operand)
            push(symbol-'0'); /* stack insert */
        else {
            /* pop two operands, perform operation, and
               push result to the stack */
            op2 = pop(); /* stack delete */
            op1 = pop();
            switch(token) {
                case plus: push(op1+op2);
                           break;
                case minus: push(op1-op2);
                           break;
                case times: push(op1*op2);
                           break;
                case divide: push(op1/op2);
                           break;
                case mod: push(op1%op2);
                           break;
            }
        }
        token = getToken(&symbol, &n);
    }
    return pop(); /* return result */
}

```

Program 3.13: Function to evaluate a postfix expression

134 Stacks And Queues

```
precedence getToken(char *symbol, int *n)
/* get the next token, symbol is the character
representation, which is returned, the token is
represented by its enumerated value, which
is returned in the function name */
*symbol = expr[(*n)++];
switch (*symbol) {
case '(' : return lparen;
case ')' : return rparen;
case '+' : return plus;
case '-' : return minus;
case '/' : return divide;
case '*' : return times;
case '%' : return mod;
case ' ' : return eos;
default : return operand; /* no error checking,
                           default is operand */
}
```

Program 3.14: Function to get a token from the input string

on their precedence. Since we must output the higher precedence operators first, we save operators until we know their correct placement. A stack is one way of doing this, but removing operators correctly is problematic. Two examples illustrate the problem.

Example 3.3 [Simple expression]: Suppose we have the simple expression $a+b*c$, which yields $abc*+$ in postfix. As Figure 3.15 illustrates, the operands are output immediately, but the two operators need to be reversed. In general, operators with higher precedence must be output before those with lower precedence. Therefore, we stack operators as long as the precedence of the operator at the top of the stack is less than the precedence of the incoming operator. In this particular example, the unstacking occurs only when we reach the end of the expression. At this point, the two operators are removed. Since the operator with the higher precedence is on top of the stack, it is removed first. $\square$

Example 3.4 [Parenthesized expression]: Parentheses make the translation process more difficult because the equivalent postfix expression will be parenthesis-free. We use as our example the expression $a*(b+c)*d$, which yields $abc+*d*$ in postfix. Figure 3.16 shows the translation process. Notice that we stack operators until we reach the

Token	Stack			Top	Output
	[0]	[1]	[2]		
<i>a</i>				-1	<i>a</i>
+	+			0	<i>a</i>
<i>b</i>	+			0	<i>ab</i>
*	+	*		1	<i>ab</i>
<i>c</i>	+	*		1	<i>abc</i>
<i>eos</i>				-1	<i>abc*+</i>

Figure 3.15: Translation of $a + b * c$ to postfix

right parenthesis. At this point we unstack until we reach the corresponding left parenthesis. We then delete the left parenthesis from the stack. (The right parenthesis is never put on the stack.) This leaves us with only the $*d$ remaining in the infix expression. Since the two multiplications have equal precedences, one is output before the d , the second is placed on the stack and removed after the d is output. $\square$

Token	Stack			Top	Output
	[0]	[1]	[2]		
<i>a</i>				-1	<i>a</i>
*	*			0	<i>a</i>
(	*	(		1	<i>a</i>
<i>b</i>	*	(		1	<i>ab</i>
+	*	(	+	2	<i>ab</i>
<i>c</i>	*	(	+	2	<i>abc</i>
)	*			0	<i>abc +</i>
*	*			0	<i>abc + *</i>
<i>d</i>	*			0	<i>abc + * d</i>
<i>eos</i>	*			0	<i>abc + * d *</i>

Figure 3.16: Translation of $a * (b + c) * d$ to postfix

The analysis of the two examples suggests a precedence-based scheme for stacking and unstacking operators. The left parenthesis complicates matters because it behaves like a low-precedence operator when it is on the stack, and a high-precedence

one when it is not. It is placed in the stack whenever it is found in the expression, but it is unstacked only when its matching right parenthesis is found. Thus, we have two types of precedence, an *in-stack precedence* (*isp*) and an *incoming precedence* (*icp*). The declarations that establish these precedences are:

```
/* isp and icp arrays -- index is value of precedence
   lparen, rparen, plus, minus, times, divide, mod, eos */
int isp[] = {0, 19, 12, 12, 13, 13, 13, 0};
int icp[] = {20, 19, 12, 12, 13, 13, 13, 0};
```

Notice that we are now using the stack to store the mnemonic for the token. That is, the data type of the stack elements is *precedence*. Since the value of a variable of an enumerated type is simply the integer corresponding to the position of the value in the enumerated type, we can use the mnemonic as an index into the two arrays. For example, *isp[plus]* is translated into *isp[2]*, which gives us an in-stack precedence of 12. The precedences are taken from Figure 3.12, but we have added precedences for the left and right parentheses and the *eos* marker. We give the right parenthesis an in-stack and incoming precedence (19) that is greater than the precedence of any operator in Figure 3.12. We give the left parenthesis an instack precedence of zero, and an incoming precedence (20) greater than that of the right parenthesis. In addition, because we want unstacking to occur when we reach the end of the string, we give the *eos* token a low precedence (0). These precedences suggest that we remove an operator from the stack only if its instack precedence is greater than or equal to the incoming precedence of the new operator.

The function *postfix* (Program 3.15) converts an infix expression into a postfix one using the process just discussed. This function invokes a function, *print_token*, to print out the character associated with the enumerated type. That is, *print_token* reverses the process used in *get_token*.

Analysis of *postfix*: Let n be the number of tokens in the expression. $\Theta(n)$ time is spent extracting tokens and outputting them. Besides this, time is spent in the two **while** loops. The total time spent here is $\Theta(n)$ as the number of tokens that get stacked and unstacked is linear in n . So, the complexity of function *postfix* is $\Theta(n)$. $\square$

EXERCISES

- Write the postfix form of the following expressions:

- $a * b * c$
- $-a + b - c + d$
- $a * -b + c$
- $(a + b) * d + e / (f + a * d) + c$

```

void postfix(void)
/* output the postfix of the expression. The expression
   string, the stack, and top are global */
char symbol;
precedence token;
int n = 0;
int top = 0; /* place eos on stack */
stack[0] = eos;
for (token = getToken(&symbol, &n); token != eos;
     token = getToken(&symbol, &n)) {
    if (token == operand)
        printf("%c", symbol);
    else if (token == rparen) {
        /* unstack tokens until left parenthesis */
        while (stack[top] != lparen)
            printToken(pop());
        pop(); /* discard the left parenthesis */
    }
    else {
        /* remove and print symbols whose isp is greater
           than or equal to the current token's icp */
        while (isp[stack[top]] >= icp[token])
            printToken(pop());
        push(token);
    }
}
while ( (token = pop()) != eos)
    printToken(token);
printf("\n");
}

```

Program 3.15: Function to convert from infix to postfix

- (e) $a \&\& b \parallel c \parallel ! (e > f)$ (assuming C precedence)
- (f) $!(a \&\& !((b < c) \parallel (c > d))) \parallel (c < e)$
2. Write the *print_token* function used in *postfix* (Program 3.15).
3. Use the precedences of Figure 3.12 together with those for '(', ')', and \0 to answer the following:
 - (a) In the *postfix* function, what is the maximum number of elements that can be

138 Stacks And Queues

- on the stack at any time if the input expression, *expr*, has *n* operators and an unlimited number of nested parentheses?
- (b) What is the answer to (a) if *expr* has *n* operators and the depth of the nesting of parentheses is at most six?
- Rewrite the *eval* function so that it evaluates the unary operators + and -.
 - § Rewrite the *postfix* function so that it works with the following operators, besides those used in the text: &&, !, <<, >>, <=, !=, <, >, <=, and >=. (Hint: Write the equation so that the operators, operands, and parentheses are separated with a space, for example, $a + b > c$. Then review the functions in `<string.h>`.)
 - Another expression form that is easy to evaluate and is parenthesis-free is known as prefix. In prefix notation, the operators precede their operands. Figure 3.17 shows several infix expressions and their prefix equivalents. Notice that the order of operands is the same in infix and prefix.

Infix	Prefix
$a * b / c$	$/*abc$
$a / b - c + d * e - a * c$	$-+ - / abc * de * ac$
$a * (b + c) / d - g$	$-/*a + bcdg$

Figure 3.17: Infix and postfix expressions

- Write the prefix form of the expressions in Exercise 1.
- Write a C function that evaluates a prefix expression, *expr*. (Hint: Scan *expr* from right to left.)
- Write a C function that transforms an infix expression, *expr*, into its prefix equivalent.

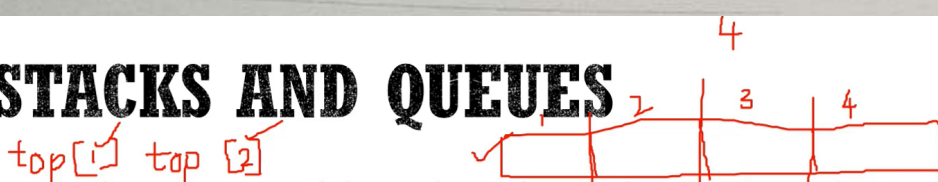
What is the time complexity of your functions for (b) and (c)? How much space is needed by each of these functions?

- Write a C function that transforms a prefix expression into a postfix one. Carefully state any assumptions you make regarding the input. How much time and space does your function take?
- Write a C function that transforms a postfix expression into a prefix one. How much time and space does your function take?
- Write a C function that transforms a postfix expression into a fully parenthesized infix expression. A fully parenthesized expression is one in which all the subexpressions are surrounded by parentheses. For example, $a + b + c$ becomes

- $((a+b)+c)$. Analyze the time and space complexity of your function.
10. Write a C function that transforms a prefix expression into a fully parenthesized infix expression. Analyze the time and space complexity of your function.
 11. § Repeat Exercise 5, but this time transform the infix expression into prefix.

3.7 MULTIPLE STACKS AND QUEUES

MULTIPLE STACKS AND QUEUES

- 
- In multiple stacks, we examine only sequential mappings of stacks into an array. The array is one dimensional which is `memory[MEMORY_SIZE]`. Assume n stacks are needed, and then divide the available memory into n segments. The array is divided in proportion if the expected sizes of the various stacks are known. Otherwise, divide the memory into equal segments.
 - Assume that i refers to the stack number of one of the n stacks. To establish this stack, create indices for both the bottom and top positions of this stack. `boundary[i]` points to the position immediately to the left of the bottom element of stack i , `top[i]` points to the top element. Stack i is empty iff `boundary[i] = top[i]`.

```
#define MEMORY_SIZE 100 /* size of memory */
#define MAX_STACKS 10 /* max number of stacks plus 1 */
/* global memory declaration */
element memory[MEMORY_SIZE];
int top[MAX_STACKS];
int boundary[MAX_STACKS];
int n; /* number of stacks entered by the user */
```

To divide the array into roughly equal segments we use the following code:

```
top[0] = boundary[0] = -1;
for (j = 1; j < n; j++)
    top[j] = boundary[j] = (MEMORY_SIZE/n)*j;
boundary[n] = MEMORY_SIZE-1;
```


140 Stacks And Queues

Figure 3.18 shows this initial configuration. In the figure, n is the number of stacks entered by the user, $n < \text{MAX_STACKS}$, and $m = \text{MEMORY_SIZE}$. Stack i can grow from $\text{boundary}[i] + 1$ to $\text{boundary}[i + 1]$ before it is full. Since we need a boundary for the last stack, we set $\text{boundary}[n]$ to $\text{MEMORY_SIZE} - 1$. Programs 3.16 and 3.17 implement the add and delete operations for this representation.

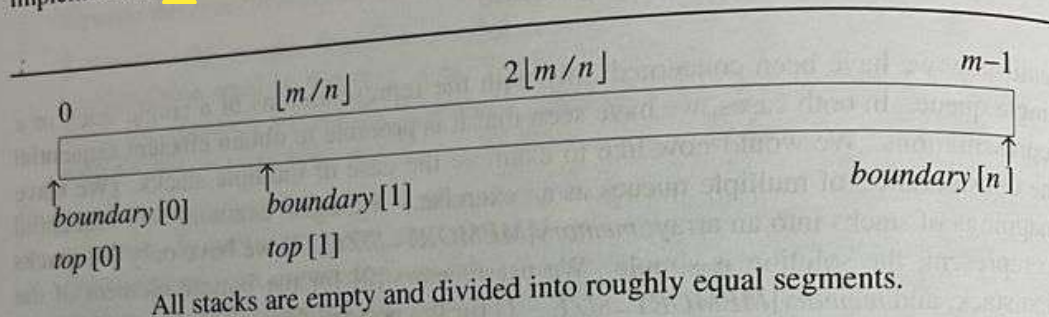


Figure 3.18: Initial configuration for n stacks in memory $[m]$.

```
void push(int i, element item)
/* add an item to the ith stack */
if (top[i] == boundary[i+1])
    stackFull(i);
memory[++top[i]] = item;
}
```

Program 3.16: Add an item to the i th stack

The *push* (Program 3.16) and *pop* (Program 3.17) functions for multiple stacks appear to be as simple as those we used for the representation of a single stack. However, this is not really the case because the $\text{top}[i] == \text{boundary}[i+1]$ condition in *push* implies only that a particular stack ran out of memory, not that the entire memory is full. In fact, there may be a lot of unused space between other stacks in array *memory* (see Figure 3.19). Therefore, we create an error recovery function, *stackFull*, which determines if there is any free space in memory. If there is space available, it should shift the stacks so that space is allocated to the full stack.

There are several ways that we can design *stackFull* so that we can add elements to this stack until the array is full. We outline one method here. Other methods are discussed in the exercises. We can guarantee that *stackFull* adds elements as long as there is free space in array *memory* if we:

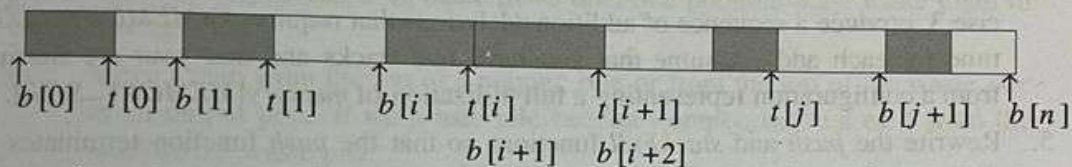
- (1) Determine the least, j , $i < j < n$, such that there is free space between stacks j and j


```

element pop(int i)
{
    /* remove top element from the ith stack */
    if (top[i] == boundary[i])
        return stackEmpty(i);
    return memory[top[i]--];
}

```

★ Program 3.17: Delete an item from the i th stack



$b = \text{boundary}, t = \text{top}$

★ Figure 3.19: Configuration when stack i meets stack $i + 1$, but the memory is not full

Method to design stackFull

- Determine the least, j , $i < j < n$, such that there is free space between stacks j and $j+1$. That is, $\text{top}[j] < \text{boundary}[j+1]$. If there is a j , then move stacks $i+1, i+2, \dots, j$ one position to the right (treating $\text{memory}[0]$ as leftmost and $\text{memory}[\text{MEMORY_SIZE} - 1]$ as rightmost). This creates a space between stacks i and $i+1$.
- If there is no j as in (1), then look to the left of stack i . Find the largest j such that $0 \leq j \leq i$ and there is space between stacks j and $j+1$ i.e., $\text{top}[j] < \text{boundary}[j+1]$. If there is a j , then move stacks $j+1, j+2, \dots, i$ one space to the left. This also creates space between stacks i and $i+1$.
- If there is no j satisfying either condition (1) or condition (2), then all MEMORY_SIZE spaces of memory are utilized and there is no free space. In this case stackFull terminates with an error message.